

Sync Conflicts

In a large-scale distributed storage system like Google Drive, sync conflicts are unavoidable.

A sync conflict occurs when:

- Two or more users modify the same file simultaneously
- Different devices update the same file before synchronization completes

Handling conflicts correctly is extremely important to ensure:

- Data consistency
- Reliability
- Good user experience

Conflict Resolution Strategy

Our system follows a simple conflict resolution rule:

First Processed Version Wins

The version that reaches and gets processed by the server first becomes the latest official version.

Any later update to the same file is considered a conflict.

Example of Sync Conflict

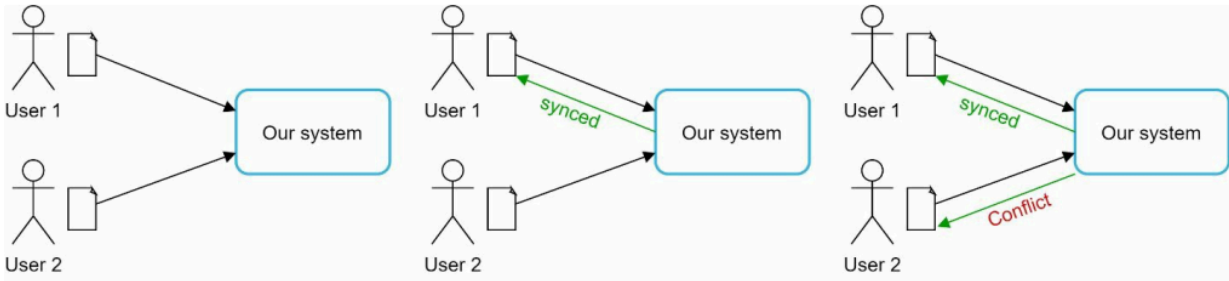


Figure demonstrates a sync conflict scenario.

Conflict Flow

1. User 1 edits a file
2. User 2 edits the same file at nearly the same time
3. User 1's update reaches the server first
4. User 1's version becomes the latest server version
5. User 2's update arrives later
6. The system detects a version mismatch and raises a sync conflict

As a result:

- User 1's update succeeds
- User 2 receives a conflict notification

Conflict Resolution for User 2

To avoid accidental data loss, the system preserves both versions of the file.

The client application presents:

- User 2's local modified version
- The latest server version

This allows the user to decide how to resolve the conflict.

Conflict Resolution UI



SystemDesignInterview



SystemDesignInterview_user 2_conflicted_copy_2019-05-01

Figure illustrates how the client displays both copies of the conflicted file.

The user can choose one of the following actions:

Merge Both Versions

Combine changes manually.

Useful for:

- Text documents
- Notes
- Source code

Override Server Version

Replace the latest server copy with the local copy.

Discard Local Changes

Keep the server version and remove local edits.

Challenges in Real-Time Collaborative Editing

Keeping documents synchronized becomes significantly harder when:

- Multiple users edit simultaneously
- Changes happen continuously
- Users work from different regions

Systems like:

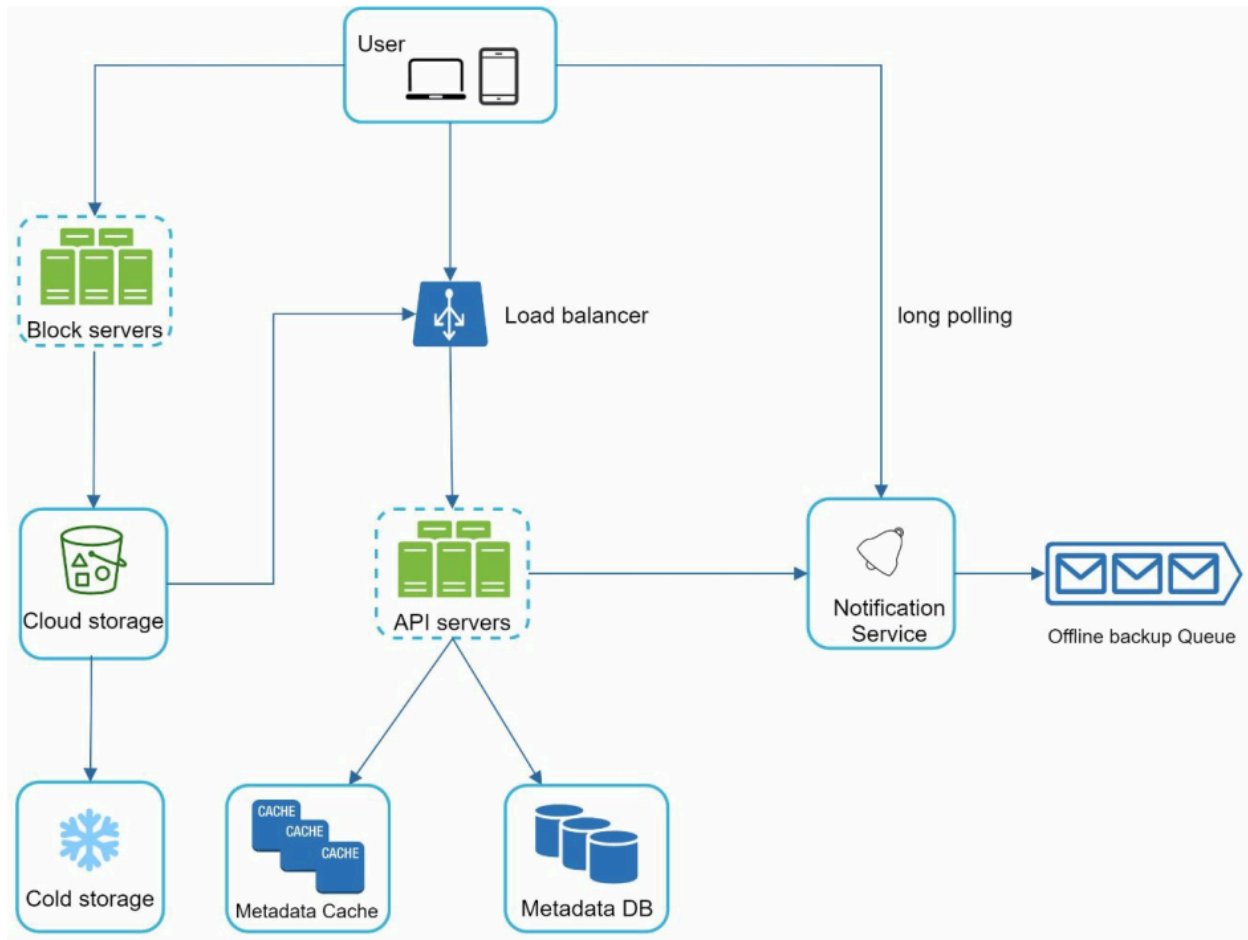
- Google Docs
- Microsoft OneDrive

use advanced synchronization algorithms such as:

- Operational Transformation (OT)
- Conflict-free Replicated Data Types (CRDT)

These are outside the scope of the current design.

High-Level Design



The improved Google Drive architecture is shown in **Figure**.

This design introduces several distributed components to support:

- File synchronization
- Notifications
- Scalability
- Reliability
- Conflict handling

High-Level Google Drive Architecture

Figure illustrates the overall distributed cloud storage architecture and interactions among major components.

The architecture contains:

- User clients
- Block servers
- Cloud storage
- Metadata services
- Notification services
- Offline synchronization mechanisms

System Components

User

Users access the system through:

- Web browsers
- Mobile applications
- Desktop applications

Users can:

- Upload files
- Download files
- Share files
- Synchronize files
- Receive notifications

Block Servers

Block servers handle file uploads efficiently.

Instead of storing a file as one large object:

- Files are split into smaller blocks
- Each block is uploaded independently

This approach is called:

Block-Level Storage

Benefits:

- Faster uploads
- Better resumability
- Efficient synchronization
- Reduced bandwidth usage

Block Storage Mechanism

Each file is divided into multiple blocks.

Each block:

- Has a unique hash value
- Is stored independently
- Can be reused if unchanged

Dropbox uses a similar strategy with:

Max Block Size = 4 MB

Advantages of Block-Level Storage

Faster Synchronization

Only modified blocks need synchronization instead of the entire file.

Efficient Deduplication

Identical blocks can be shared across files.

Resumable Uploads

If upload fails:

- Only failed blocks are retried
- Previously uploaded blocks remain intact

Cloud Storage

Actual file blocks are stored in distributed cloud object storage such as:

- [Amazon S3](#)

The storage layer provides:

- Durability
- Replication
- High availability
- Scalability

Cold Storage

Cold storage is used for:

- Rarely accessed files
- Archived data
- Old revisions

Characteristics:

- Lower storage cost
- Higher retrieval latency

Examples include:

- AWS Glacier
- Archival object storage systems

Load Balancer

The load balancer distributes incoming traffic evenly among API servers.

Responsibilities:

- Traffic routing
- Fault tolerance
- High availability
- Horizontal scalability

API Servers

API servers handle almost all business logic except direct file block uploads.

Responsibilities include:

- User authentication
- File metadata management
- Sharing permissions
- Revision tracking
- Sync coordination
- Conflict detection

API servers are stateless to allow horizontal scaling.

Metadata Database

The metadata database stores:

- User information
- File metadata
- File ownership
- Block references
- File versions
- Sharing permissions

Important note:

- Actual file content is NOT stored in the metadata database.

Only metadata is stored.

Metadata Cache

Frequently accessed metadata is cached in memory.

Benefits:

- Faster retrieval
- Reduced database load
- Lower latency

Examples of cached data:

- File paths
- User sessions
- Sharing permissions
- Revision info

Notification Service

The notification service follows a:

Publish-Subscribe (Pub/Sub) Model

When a file changes:

- Relevant users receive notifications
- Clients pull the latest updates

Notifications are triggered for:

- File edits
- File deletion

- File sharing
- Permission updates

Real-Time Synchronization

When User A edits a file:

1. Metadata is updated
2. Notification service publishes an event
3. Other devices/users receive update notifications
4. Clients fetch latest changes automatically

This enables near real-time synchronization.

Offline Backup Queue

Sometimes clients are offline when updates occur.

To handle this scenario:

- Updates are stored temporarily in an offline backup queue

When the client reconnects:

- Pending updates are replayed
- Synchronization resumes automatically

Benefits:

- Reliable sync
- No missed updates
- Better offline support

Why This Architecture Works

The distributed architecture provides:

Requirement	Solution
Large file uploads	Block storage
Fast synchronization	Block-level sync
Fault tolerance	Replication
Scalability	Distributed services
Offline support	Backup queue
Real-time updates	Notification service
Conflict handling	Version comparison

Summary

The improved Google Drive architecture introduces:

- Distributed storage
- Block-level uploads
- Metadata separation
- Notification-based synchronization
- Conflict handling mechanisms
- Offline synchronization support

This architecture supports:

- Millions of users
- Petabyte-scale storage
- Real-time synchronization
- Reliable distributed collaboration